

Genomic Mutation Tracker — Complete Project Guide

Everything you need to understand, explain, and defend this project

TABLE OF CONTENTS

1. MySQL — can you close it without saving?
 2. Runtime vs Compile time — what went wrong
 3. Why we switched to CSV + Python
 4. New file: patient.h
 5. New file: db.h
 6. New file: db.cpp
 7. New file: insert_db.py
 8. Changes in main.cpp
 9. The complete flow from start to finish
 10. Git commands reminder
 11. How to explain this project in an interview
-

1. CAN YOU CLOSE MYSQL WITHOUT SAVING?

Yes, close it anytime. MySQL is a **server** running in the background on your computer. It saves data automatically the moment you INSERT or CREATE something. There is no "Save" button in MySQL like Word or Excel.

```
You ran: python insert_db.py
Python inserted 100 patients into MySQL
MySQL saved them to disk automatically
You close Workbench → data is still there
You restart computer → data is still there
You open Workbench tomorrow → data is still there
```

MySQL data lives in a special folder on your C: drive managed by MySQL Server. You never need to manually save it.

2. RUNTIME vs COMPILE TIME — WHAT WENT WRONG

This is an important concept. There are two phases when running a program:

Compile time = when g++ converts your .cpp code into .exe **Runtime** = when you actually run the .exe

Files involved:

```
.h files    → used at COMPILE TIME (declarations)
.lib files  → used at COMPILE TIME (tells linker where functions are)
.dll files  → used at RUNTIME (actual code loaded when program runs)
```

When we tried `-lmysql`, g++ looked for `libmysql.lib` to link at compile time. But it found `libmysql.dll` instead — a runtime file. g++ said:

```
libmysql.dll: file not recognized
```

Because `.dll` is not a compile-time file. It's loaded by Windows automatically when your .exe runs, not during compilation.

Then we tried `mysqlclient.lib` which IS a compile-time file. But:

```
unhandled machine type 0x8664
```

This means:

```
mysqlclient.lib = compiled for 64-bit (0x8664 = x86_64)
your g++        = MinGW 32-bit compiler
32-bit compiler cannot link 64-bit library
incompatible!
```

Like trying to plug a 3-pin plug into a 2-pin socket. Same electricity, incompatible shape.

The fix: Use Python for MySQL because Python's connector handles all of this internally. You just write simple Python code and it works regardless of architecture.

3. WHY CSV + PYTHON (POLYGLOT ARCHITECTURE)

Instead of C++ → MySQL directly, we did:

```
C++ → writes CSV files → Python reads CSV → inserts into MySQL
```

This is called **polyglot architecture** — using multiple languages, each for what it's best at:

```
C++    → fast string algorithms (KMP, Rabin-Karp)
        no language beats C++ for raw speed
Python → easy database connectivity
        mysql-connector-python installs in 1 command
        no architecture issues
```

This is how real companies work too:

```
Netflix:  Java for backend, Python for ML, SQL for analytics
Uber:     Go for microservices, Python for data pipelines
Google:   C++ for core systems, Python for tooling
```

Interview answer: "I used C++ for core algorithms because it gives maximum performance for string processing at genome scale. For database insertion I used Python's mysql-connector because linking MySQL's 64-bit library with MinGW's 32-bit g++ on Windows causes architecture incompatibility. This is a common real-world constraint and polyglot architecture is the industry standard solution."

4. NEW FILE: patient.h

Why we created it:

Originally Patient struct was defined inside `main.cpp`. But when we created `db.cpp`, it also needed to know what a Patient is. Two options:

```
Option A: copy the struct into db.cpp too  ← BAD, duplicate code
Option B: move struct to its own .h file   ← GOOD, single source of truth
```

We chose Option B. Now:

```
patient.h  → defines Patient struct ONCE
main.cpp   → #include "patient.h"
db.cpp     → #include "patient.h"
db.h       → #include "patient.h"
```

Everyone reads from one place. If you add a field to Patient, you change it in ONE file and everywhere gets the update automatically.

The code:

```
cpp
```

```

#ifndef PATIENT_H
#define PATIENT_H

#include <string>
#include <vector>
#include <map>
using namespace std;

struct Patient {
    string id;           // "P001"
    int age;             // 34
    string dna;          // "ATCGGTAGCT..."

    // KMP results
    int totalMutations;  // total count found
    map<string, vector<int>> mutationMap; // "GGT" → [23, 145, 302]

    // Rabin-Karp results
    map<string, vector<int>> rkMutationMap; // same structure
    int rkTotalMutations;

    // statistics
    int uniqueMutations; // how many different types found
    double mutationDensity; // mutations per 1000 chars
    double riskScore; // computed risk score
};

#endif

```

Why `map<string, vector<int>>`:

```

map    → links mutation name to its positions
string → "GGT" (mutation name)
vector<int> → [23, 145, 302] (all positions where found)

So mutationMap["GGT"] = [23, 145, 302]
means GGT was found at positions 23, 145, and 302

```

5. NEW FILE: db.h

Just the header file declaring what `db.cpp` contains:

```
cpp
```

```

#ifndef DB_H
#define DB_H

#include "patient.h"
#include <unordered_map>
using namespace std;

void insertToMySQL(vector<Patient>& patients,
                  unordered_map<string, int>& freqMap);

#endif

```

Takes two parameters:

```

vector<Patient>&           → all 100 patients with their results
unordered_map<string, int>& → global mutation frequencies
                        "GGT" → 1637

```

The **&** means pass by reference — don't copy the data, just point to it. Since patients vector is large (100 patients × 1000 char DNA each), copying would be wasteful.

6. NEW FILE: db.cpp

This file writes 3 CSV files from C++ results. Originally it was supposed to connect to MySQL directly but we switched to CSV due to architecture issues (explained above).

Three CSV files it creates:

```

results.csv           → one row per patient (stats + risk score)
mutation_results.csv → one row per mutation occurrence (patient, mutation,
position)
frequencies.csv       → one row per mutation type (global count)

```

Writing results.csv:

```
cpp
```

```

ofstream pFile("results.csv");
pFile << "patient_id,age,dna_length,total_mutations,"
        << "unique_mutations,mutation_density,risk_score\n";

for (auto& p : patients) {
    pFile << p.id          << ","
          << p.age         << ","
          << p.dna.length() << ","
          << p.totalMutations << ","
          << p.uniqueMutations << ","
          << p.mutationDensity << ","
          << p.riskScore      << "\n";
}
pFile.close();

```

First line writes the header (column names). Then for each patient we write one line with all their stats separated by commas. This creates:

```

patient_id,age,dna_length,total_mutations,unique_mutations,mutation_density,risk_score
P001,58,1000,21,4,21.0,20.8
P002,67,1000,26,5,26.0,25.8
...

```

Writing mutation_results.csv:

```

cpp

ofstream mFile("mutation_results.csv");
mFile << "patient_id,mutation_name,position,algorithm\n";

for (auto& p : patients) {
    for (auto& entry : p.mutationMap) {
        for (int pos : entry.second) {
            mFile << p.id          << ","
                  << entry.first << ","
                  << pos         << ","
                  << "KMP"       << "\n";
        }
    }
}

```

Three nested loops:

outer loop → each patient
middle loop → each mutation type found in this patient
inner loop → each position where that mutation was found

Patient P001:

GGT found at 92, 167, 246...

→ writes:

P001,GGT,92,KMP

P001,GGT,167,KMP

P001,GGT,246,KMP

ATCGG found at 726

→ writes:

P001,ATCGG,726,KMP

Writing frequencies.csv:

cpp

```
ofstream fFile("frequencies.csv");  
fFile << "mutation_name,total_count\n";  
  
for (auto& entry : freqMap) {  
    fFile << entry.first << ","  
        << entry.second << "\n";  
}
```

Simple — just writes each mutation and its global count:

```
mutation_name,total_count  
GGT,1637  
TAGG,448  
CGGA,427  
ATCGG,132  
TTTCGA,36
```

7. NEW FILE: insert_db.py

Reads the 3 CSV files and inserts into MySQL. Python chosen because mysql-connector-python works seamlessly regardless of 32/64 bit issues.

Connection:

python

```
conn = mysql.connector.connect(  
    host="localhost",      # MySQL is on your own computer  
    user="root",          # default MySQL admin user  
    password="...",       # your MySQL password  
    database="genomic_tracker" # which database to use  
)  
cursor = conn.cursor()    # cursor = pen to write SQL queries
```

Think of `conn` as opening the door to MySQL. `cursor` is the pen you use to write queries inside.

Clear old data first:

```
python  
  
cursor.execute("DELETE FROM mutation_results") # child table first  
cursor.execute("DELETE FROM patients")         # parent table second  
cursor.execute("DELETE FROM mutation_frequencies")  
conn.commit()
```

Why delete `mutation_results` BEFORE `patients`? Because of FOREIGN KEY:

```
mutation_results.patient_id → references → patients.patient_id
```

```
If you delete patients first:
```

```
MySQL says "mutation_results still references these patients!"
```

```
ERROR: foreign key constraint fails
```

```
Delete children first (mutation_results)
```

```
Then delete parent (patients)
```

```
No constraint violation
```

Reading CSV and inserting:

```
python
```

```

with open("results.csv", "r") as f:
    reader = csv.DictReader(f)    # reads CSV with column names as keys
    for row in reader:
        cursor.execute("""
            INSERT INTO patients VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
        """, (
            row["patient_id"],
            int(row["age"]),        # convert string to int
            int(row["dna_length"]),
            int(row["total_mutations"]),
            int(row["unique_mutations"]),
            float(row["mutation_density"]), # convert string to float
            float(row["risk_score"])
        ))
conn.commit() # save all inserts permanently

```

`csv.DictReader` reads each row as a dictionary:

```

row = {
    "patient_id": "P001",
    "age": "58",                ← everything is string from CSV
    "total_mutations": "21",
    ...
}

```

We convert strings to int/float because MySQL expects numbers not strings.

`%s` are placeholders — Python fills them with actual values safely (prevents SQL injection attacks).

`conn.commit()` is like pressing Save — without it, inserts exist temporarily in memory but never reach disk.

8. CHANGES IN main.cpp

Change 1: Added includes

```

cpp

#include <unordered_map> // for freqMap
#include <queue>          // for priority_queue top-K
#include <chrono>         // for timing
#include "patient.h"     // Patient struct (moved here)
#include "db.h"          // insertToMySQL function

```

Change 2: Removed Patient struct Moved to `patient.h`. `main.cpp` just includes it now.

Change 3: Added timing around search loops

```
cpp

auto kmpStart = chrono::high_resolution_clock::now();
// KMP runs
auto kmpEnd = chrono::high_resolution_clock::now();
double kmpTime = chrono::duration<double, milli>(kmpEnd - kmpStart).count();
```

`chrono::high_resolution_clock::now()` captures exact time. Difference between end and start = how long it took. Converted to milliseconds.

Change 4: Added computeStats function

```
cpp

void computeStats(Patient& p) {
    p.uniqueMutations = p.mutationMap.size();
    p.mutationDensity = (double)p.totalMutations / p.dna.length() * 1000.0;
    p.riskScore = (p.totalMutations * 0.5)
        + (p.uniqueMutations * 1.0)
        + (p.mutationDensity * 0.3);
}
```

Three statistics computed:

uniqueMutations = `mutationMap.size()` = number of keys in the map = number of different mutation types found. If GGT and ATCGG were found, size = 2.

mutationDensity = mutations per 1000 characters:

```
formula: (total / dna_length) × 1000
example: (21 / 1000) × 1000 = 21.0
means: 21 mutations per 1000 characters
```

riskScore = weighted formula:

$\text{totalMutations} \times 0.5 \rightarrow$ raw count matters but less
repeating same mutation isn't as dangerous
as new mutation types

$\text{uniqueMutations} \times 1.0 \rightarrow$ different types = more dangerous
highest weight

$\text{mutationDensity} \times 0.3 \rightarrow$ concentration matters least
spread out mutations less concerning

Example P001:

$(21 \times 0.5) + (4 \times 1.0) + (21 \times 0.3)$
 $= 10.5 + 4.0 + 6.3$
 $= 20.8$

Change 5: Added global frequency tracking

```
cpp
unordered_map<string, int> freqMap;

void computeFrequencies(vector<Patient>& patients,
                        unordered_map<string, int>& freqMap) {
    for (auto& p : patients) {
        for (auto& entry : p.mutationMap) {
            freqMap[entry.first] += entry.second.size();
        }
    }
}
```

Goes through every patient, every mutation type, adds count to global map:

```
Patient P001: GGT found 13 times → freqMap["GGT"] += 13
Patient P002: GGT found 8 times  → freqMap["GGT"] += 8
...
After all 100 patients: freqMap["GGT"] = 1637
```

Change 6: Added Top-K using priority_queue

```
cpp
```

```

void printTopK(unordered_map<string, int>& freqMap, int k) {
    priority_queue<pair<int, string>> pq;

    for (auto& entry : freqMap) {
        pq.push({entry.second, entry.first});
    }

    int rank = 1;
    while (!pq.empty() && rank <= k) {
        auto top = pq.top();
        pq.pop();
        cout << rank << ". " << top.second << " → " << top.first << "\n";
        rank++;
    }
}

```

`priority_queue` is a max-heap — highest value always at top.

We push `{frequency, name}` pairs. Since int comes first, it sorts by frequency:

```

{1637, "GGT"}    ← highest, always at top
{448, "TAGG"}
{427, "CGGA"}
{132, "ATCGG"}
{36, "TTTCGA"}  ← lowest

```

`pq.top()` gives highest. `pq.pop()` removes it. Loop k times to get top-K.

Change 7: Added MySQL insertion call

```

cpp

insertToMySQL(patients, freqMap);

```

One line at end of main. Calls db.cpp which writes 3 CSV files.

9. COMPLETE FLOW FROM START TO FINISH

1. generator.cpp runs
 - creates patients.txt (100 patients, age-correlated mutations)
2. main.cpp runs
 - reads patients.txt into `vector<Patient>`
 - KMP searches all 5 mutations in each patient DNA

- Rabin-Karp searches all 5 mutations (grouped by length)
- timing comparison printed
- computeStats() calculates density, unique, risk score
- computeFrequencies() aggregates global counts
- printTopK() shows most common mutations
- insertToMySQL() writes 3 CSV files

3. insert_db.py runs

- connects to MySQL
- clears old data
- reads results.csv → inserts into patients table
- reads mutation_results.csv → inserts into mutation_results table
- reads frequencies.csv → inserts into mutation_frequencies table

4. MySQL Workbench

- run setup.sql queries
- see top risk patients
- see age group analysis
- see window function rankings

10. GIT COMMANDS REMINDER

git add .	→ stage all changed files
git commit -m "message"	→ save snapshot with description
git push	→ send to GitHub

git status	→ see what changed
git log	→ see commit history

Always: add → commit → push

Never push .exe files (covered by .gitignore)

11. HOW TO EXPLAIN IN INTERVIEW

"Tell me about your project":

"I built a genomic mutation tracker that searches patient DNA sequences for disease-causing patterns. I implemented both KMP and Rabin-Karp algorithms in C++, benchmarked them against each other, computed patient risk scores, and stored results in MySQL for analytics using SQL window functions."

"Why two algorithms?":

"To demonstrate tradeoffs. KMP is optimal for single pattern search using its LPS array preprocessing – $O(N+M)$. Rabin-Karp uses rolling hash which is better for multiple patterns simultaneously. On our 1000-char DNA, KMP won due to lower setup overhead. At genome scale (3 billion chars, 50+ patterns), Rabin-Karp would win due to $O(1)$ rolling hash."

"Why Python for MySQL?":

"MinGW g++ on Windows is 32-bit but MySQL 8.0 is 64-bit. The .lib files are architecture-incompatible at link time. Python's mysql-connector handles this internally. This is polyglot architecture – using the right tool for each job."

"Is your data realistic?":

"It's a simulation. Mutation injection rate correlates with age to model the biological reality that older patients accumulate more DNA copying errors. I documented this limitation in the README and noted that production would use NCBI genomic datasets. The algorithms are identical regardless of data source."

"What is the risk score?":

"A weighted combination of three factors: total mutation count (weight 0.5), unique mutation types (weight 1.0, highest because diversity is more dangerous than repetition), and mutation density (weight 0.3). It's a simplified model – production would use clinically validated weights from published research."

"What SQL concepts did you use?":

"CASE statements for age group bucketing, GROUP BY for aggregation, ORDER BY for ranking, RANK() window function for patient risk ranking with ties handled correctly, AVG() OVER() for running averages across all patients, and FOREIGN KEY constraints for data integrity."

FINAL PROJECT STATS

```
Languages:      C++, Python, SQL
Files:          15 files
Algorithms:     KMP, Rabin-Karp, Rolling Hash
Data Structures: vector, map, unordered_map, priority_queue, struct
Database:       MySQL with 3 tables, 5 analytics queries
Concepts:       String matching, hashing, heaps, window functions,
                polyglot architecture, header files, file I/O
Time taken:     ~12 hours (one session!)
```

You built a complete end-to-end system in one day. That's impressive.

0